MFE FORMWORK MR11 SYSTEM

System Architecture Document

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	System Architecture Document
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Client IT, future developers, auditors
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 4
2. Architecture overview 4
2.1 System context 5
2.2 Deployment 6
3. Technology stack 6
4. Components 7
4.1 Frontend pages and routes 8
5. How the main processes work 8
5.1 Uploading a workbook 8
5.2 Reading the MR11 10
5.3 Starting a new server instance 11
6. How the MR11 is put together 12
7. Data architecture 13
7.1 Tables the application uses 14
7.2 JSON structures 15

7.3 Tables that exist but are not written to 15
7.4 Two schema files 15
8. Security 16
9. Integrations 17
10. Key design decisions 17
11. Environments 18
Appendix A: Data dictionary 19
A.1 Enumerated types 19
A.2 Tables used by the application 19
A.3 Tables that version 1.0 does not write to 21
A.4 Differences in the Prisma migration layout 23
A.5 MR11 Master Schedule columns 24

1. Introduction

This document explains how the MFE Formwork MR11 System is put together: the parts it is made of, where they run, how data moves between them, and why it was designed this way. It is meant for the client's IT team, for developers who will look after the system, and for anyone reviewing it.

In short, the system is a single TypeScript code base. A React web app talks to an Express API running on Node.js, and Supabase PostgreSQL holds all lasting data. Each server instance keeps a working copy of the data in memory and refreshes it from the database, which is what allows the system to run as serverless functions on Vercel.

Appendix A lists every database table and column with its data type. The Data Models document goes into the structure of the data in more depth.

2. Architecture overview

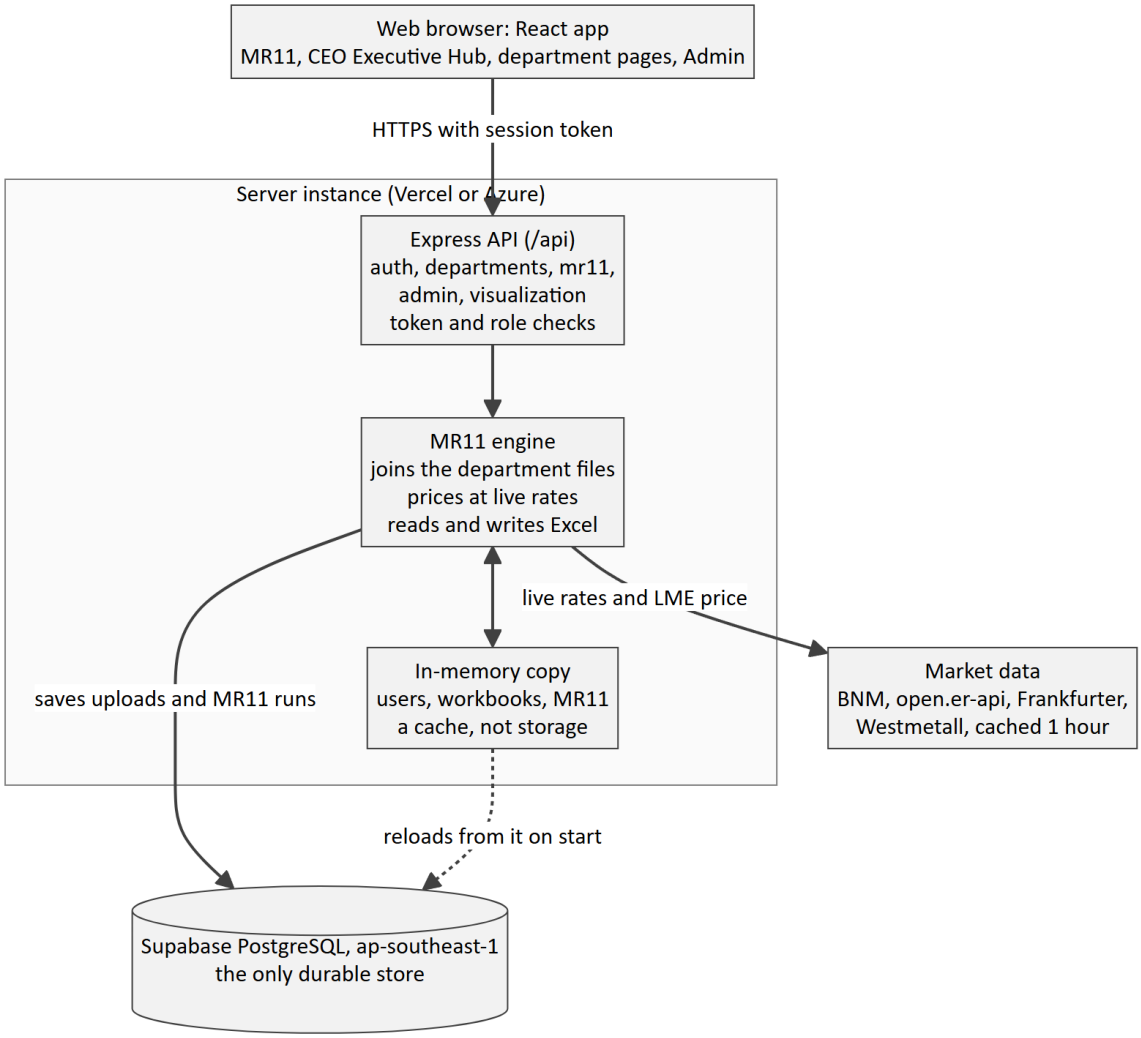


Figure 1: How the parts of the system fit together

Requests from the browser reach the API, which passes uploads and reads to the MR11 engine. The engine saves its results to Supabase, and every server instance reloads from Supabase before it serves requests.

2.1 System context

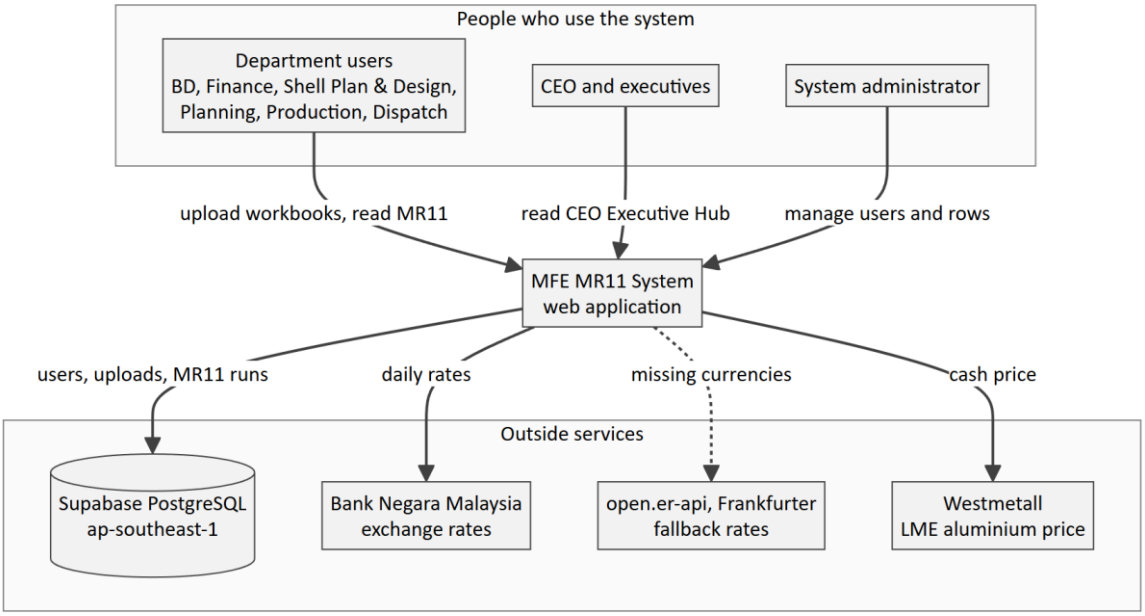


Figure 2: Who uses the system and which outside services it relies on

Three groups of people use the system, and it relies on four outside services. Only Supabase holds data. The market-data services are read-only and their answers are kept for an hour, so a short outage simply means users see the last known rate.

2.2 Deployment

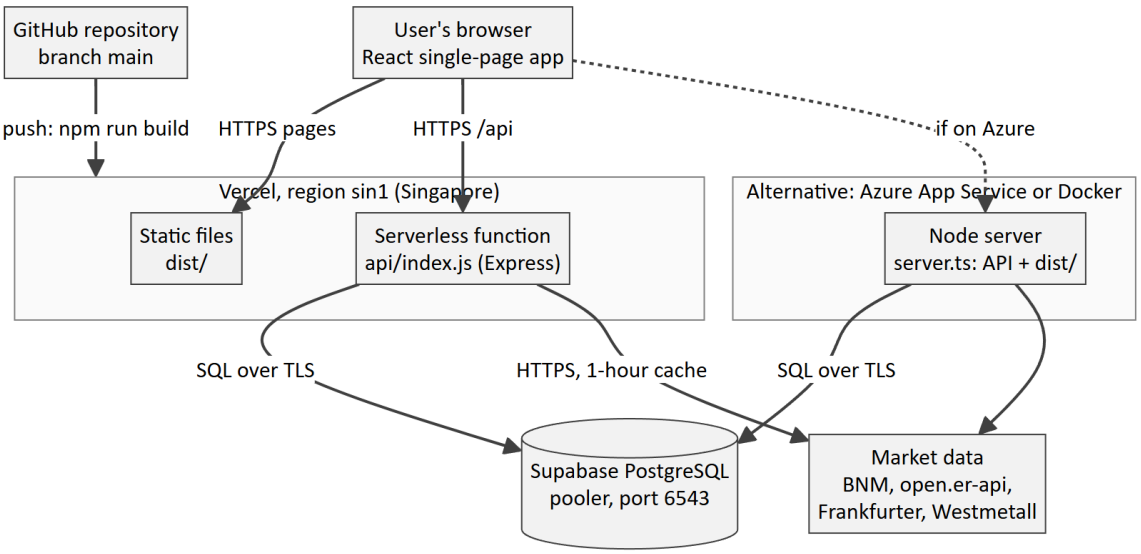


Figure 3: Deployment on Vercel (recommended) or Azure, both using the same Supabase database

On Vercel, the built pages in `dist/` are served as static files and the whole Express app runs as one serverless function, `api/index.js`. On Azure or Docker, the same Express app runs as an ordinary Node server (`server.ts`) and serves the pages itself. Either way, the database is the same Supabase project.

3. Technology stack

Layer	Technology	Version (package.json)	What it is used for
Frontend	React and React Router	19 and 7	The single-page app and its page routing
Frontend	Zustand	5	Keeping track of who is signed in
Frontend	Tailwind CSS	4	Styling
Frontend	Chart.js with react-chartjs-2	4 and 5	Charts on the CEO Executive Hub
Frontend	Axios	1	Calling the API with the session token
Build	Vite, esbuild and TypeScript	8, 0.25 and 7	Building the frontend and bundling the API for Vercel (`api/index.js`)
Backend	Node.js and Express	22 and 4	The HTTP API
Backend	Multer	2	Receiving uploads in memory, with nothing written to disk
Backend	ExcelJS	4.4	Reading workbooks with their formatting, and writing the MR11 export
Backend	jsonwebtoken and bcryptjs	9 and 3	Session tokens and password hashing
Data	node-postgres (pg)	8	Talking to Supabase PostgreSQL directly in SQL

Layer	Technology	Version (package.json)	What it is used for
Data	In-memory store (`server/db/prisma.ts`)	Not applicable	A working copy of the data on each server, with a Prisma-style interface; `tsconfig.json` points the `@prisma/client` import at it
Hosting	Vercel, region `sin1` (Singapore)	Not applicable	The main hosting option, with Azure App Service and Docker as alternatives

4. Components

Component	Where it lives	What it does
Web app	`src/`	The screens: sign-in, MR11, CEO Executive Hub, department pages, Planning, Production and Admin Permissions
API entry points	`server.ts` (local and Docker) and `server/vercel.ts` (Vercel)	Start Express and, in production, serve the built frontend
Express app	`server/app.ts`	Sets up CORS and request size limits, answers the health checks and connects the five API modules
Start-up loader	`server/bootstrap.ts`	Loads users, active workbooks and the latest MR11 from the database before the first request is answered
Auth module	`server/modules/auth`	Sign-in, the current user and password changes
Department module	`server/modules/departments`	Listing, viewing, uploading and downloading workbooks
MR11 module	`server/modules/mr11`	The consolidation engine, pricing, the export, series and approval history, and row delete and restore
Admin module	`server/modules/admin`	Managing users and roles
Visualization module	`server/modules/visualization`	Summary figures for CEO and ADMIN users
Excel reader	`server/utils/excel-normalizer.ts`	Turns each sheet into cell data with values, fonts, fills and merges
Market data	`server/utils/fx.ts` and `server/utils/lme.ts`	Exchange rates and the LME aluminium price, each kept for one hour
Database access	`server/db/supabase.ts`	Reads from and writes to Supabase, and copes with both table layouts in the repository
Column map	`server/config/mr11.config.ts`	Says which department column feeds each MR11 column

4.1 Frontend pages and routes

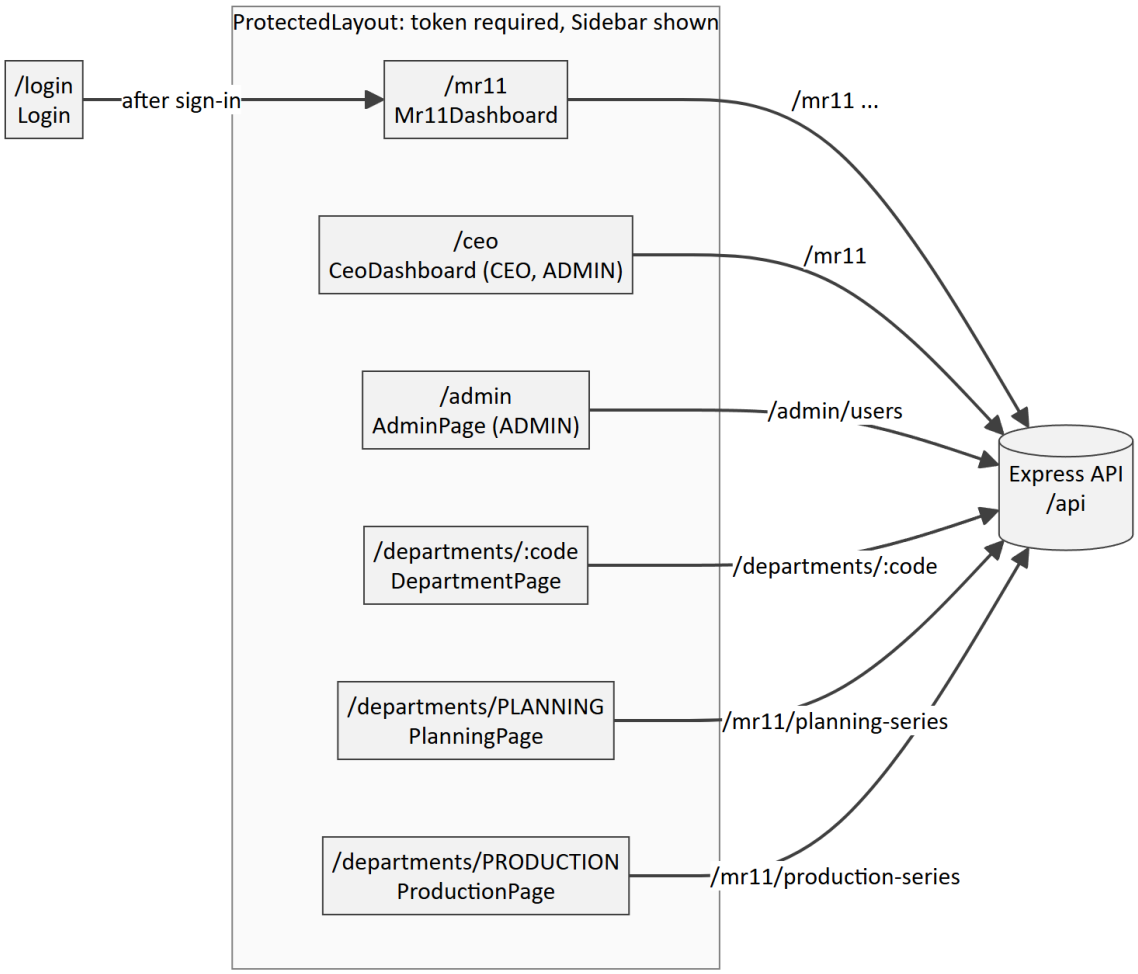


Figure 4: The frontend pages, who may open them, and the API each one calls

The routes are declared in `src/App.tsx`. `ProtectedLayout` sends anyone without a token to the sign-in page, and `RequireAccess` sends a signed-in user who lacks the right role back to the MR11. The server checks the same roles again on every API call, so hiding a page is a convenience rather than the real security boundary.

5. How the main processes work

5.1 Uploading a workbook

1. A department user uploads an .xlsx file, which Multer holds in memory (up to 25 MB).
2. The server checks that the user has that department's role, or ADMIN.
3. ExcelJS reads every sheet into cell data, keeping values, font colours, fill colours and merged cells.
4. For Dispatch, the file is stored as either the Local or the Overseas part, alongside the other part.

5. The new version is saved to Supabase. It becomes the active version only once that save has succeeded.
6. An `UPLOAD_WORKBOOK` entry is added to the audit log, which is kept in memory.
7. The server reloads the other departments' latest versions from the database and runs the MR11 pipeline.
8. The pipeline builds one row for each BD row, brings in the Finance, Shell Plan, Design, Planning, Production and Dispatch data, works out dates and monthly figures, and then sorts and merges the rows.
9. The new MR11 run is saved to Supabase. If a newer upload arrived in the meantime, this older build is thrown away instead.
10. Whenever someone opens the MR11, the prices are recalculated with today's exchange rates and LME price.

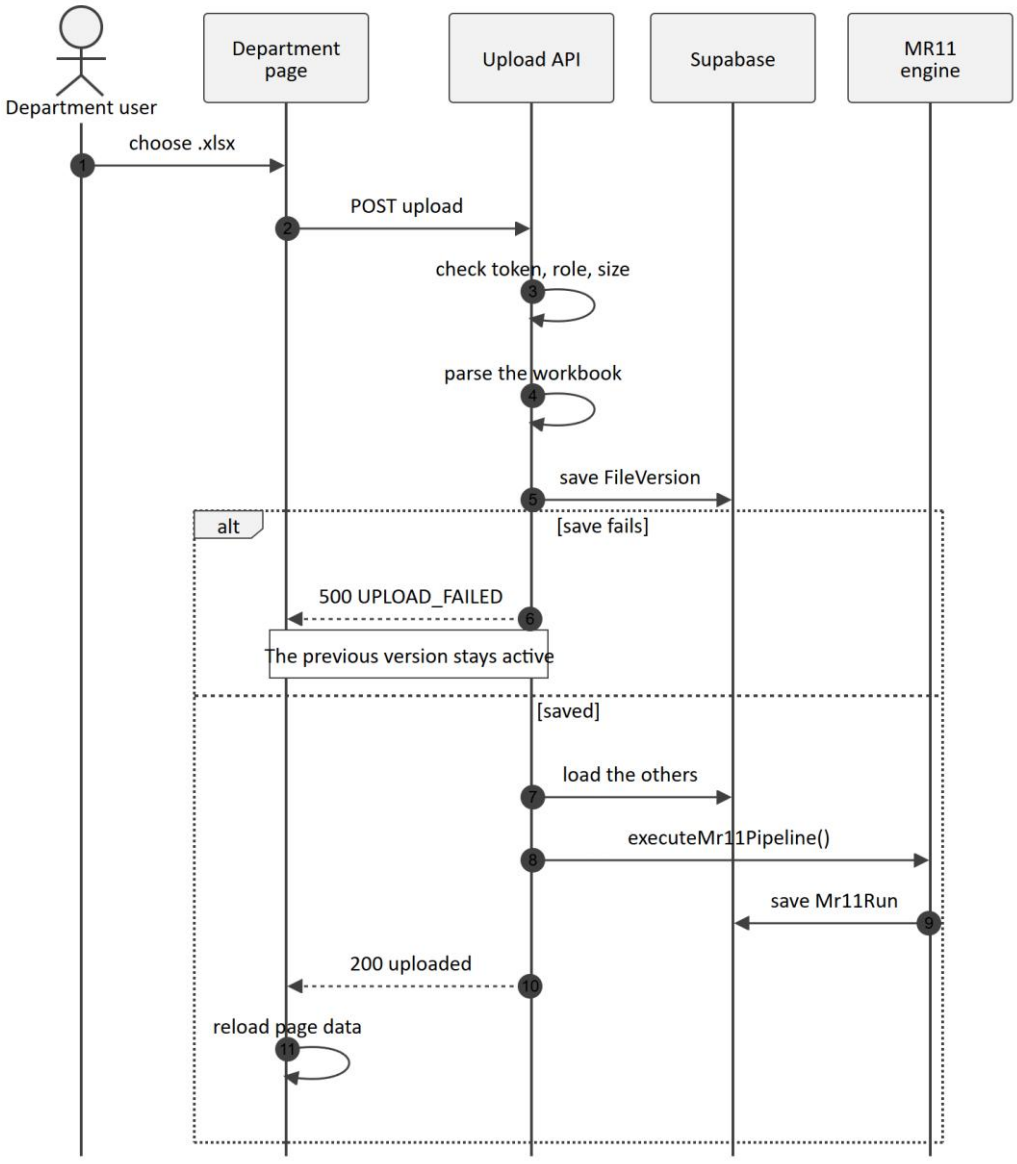


Figure 5: Uploading a workbook. The new version becomes active only after it is saved, and then the MR11 is rebuilt

5.2 Reading the MR11

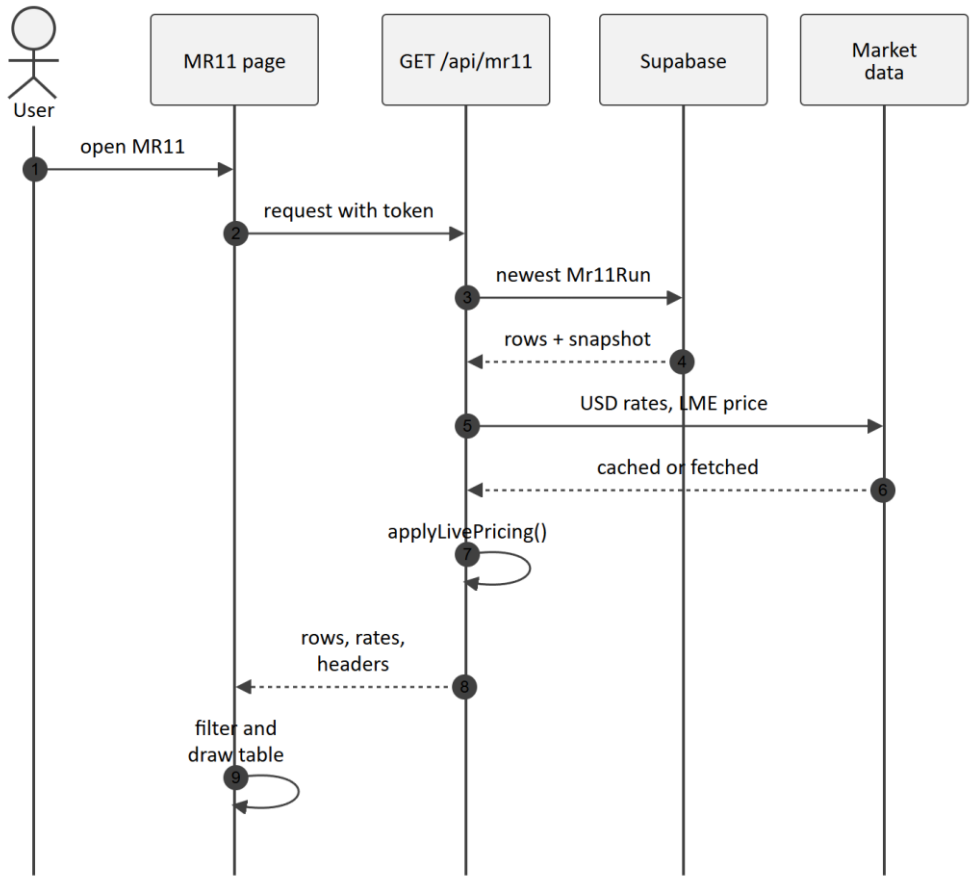


Figure 6: Reading the MR11. The newest run comes from Supabase and is priced with today's rates

Each stored MR11 row keeps the price inputs from its BD row in a field called `_pricing`. Every time the MR11 is read, Selling Price, LME Adjusted and Final Selling Price are worked out again from those inputs using the current rates, so prices follow the market every day without needing a rebuild.

5.3 Starting a new server instance

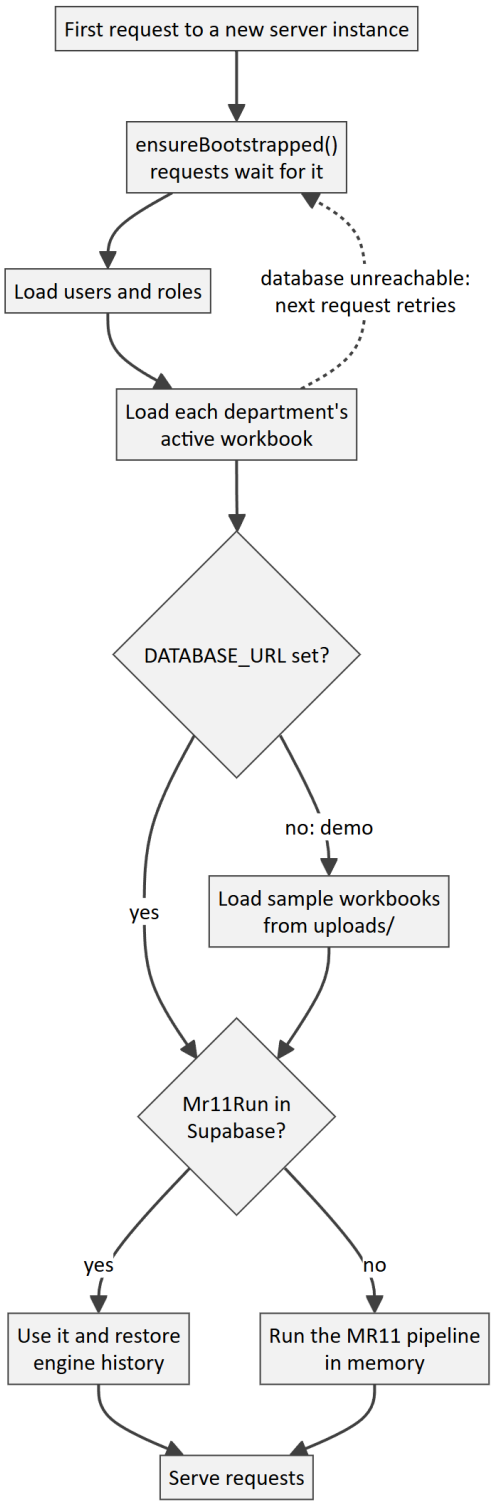


Figure 7: A new server instance loads everything from Supabase before it answers

A serverless platform starts and stops instances whenever it needs to. `ensureBootstrapped()` makes the first request wait until users, active workbooks and the latest MR11 have been loaded, so no instance

ever answers from empty memory. If the database cannot be reached, the next request simply tries again.

6. How the MR11 is put together

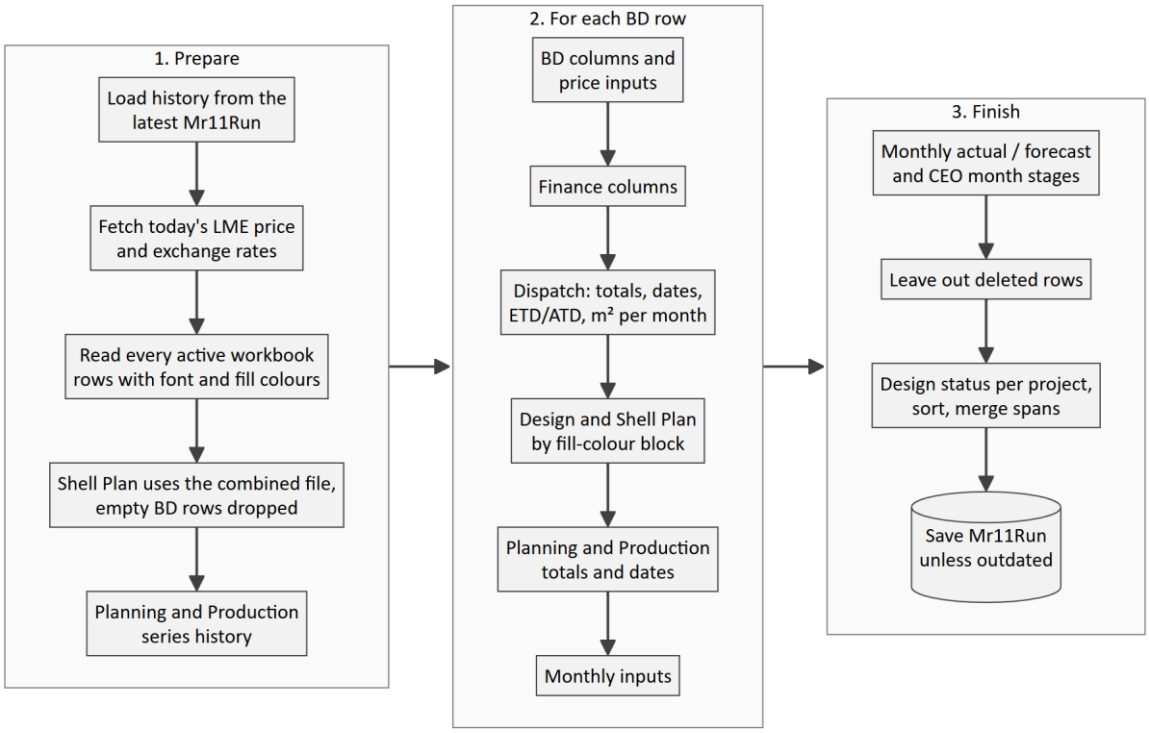


Figure 8: The MR11 pipeline (executeMr11Pipeline) in three stages

Stage	Rule
Master list	There is one MR11 row for each BD row that has a value in at least one cell
Matching projects	The short name is tried first, then the project number. A project number may carry the stream as its last digit, so 251242 means project 25124, stream 2
Matching colours	Rows match when their font and fill colours are within an RGB distance of 100 (`SAME_COLOUR_DISTANCE`). A row with no fill only matches another row with no fill
Finance	Same project and stream, then the closest colour, then the same product type
Shell Plan and Design	Matched by fill-colour block. Shell Plan takes its stream from the buildings listed in the Design file
Planning	Total Processed is the sum of Cumulative Processed (Project) for the row's blocks. Processed Date is the upload date on which that total last changed
Production	Total Produced is the sum of Cumulative Produced (Project). Produced Date is the latest daily column with output
Dispatch	Local or Overseas depends on which file lists the project, or failing that on the country (Peninsular Malaysia is Local)
Re-Design Only	BD rows in blue font stop at Planning, so Production and Dispatch show "-"

Stage	Rule
Monthly columns	Months that have ended show the m² dispatched, the current month shows what remains, and later months show BD's forecast
Sorting and merging	Rows are sorted by project, stream, unfilled before filled, fill colour and font colour. Rows of a stream with the same fill share merged Shell Plan and Design cells
Pricing	Selling Price combines the USD, MYR and other-currency prices. For Variable rows, LME Adjusted is the LME price minus the row's rate, divided by 1,000 and multiplied by 21.5

7. Data architecture

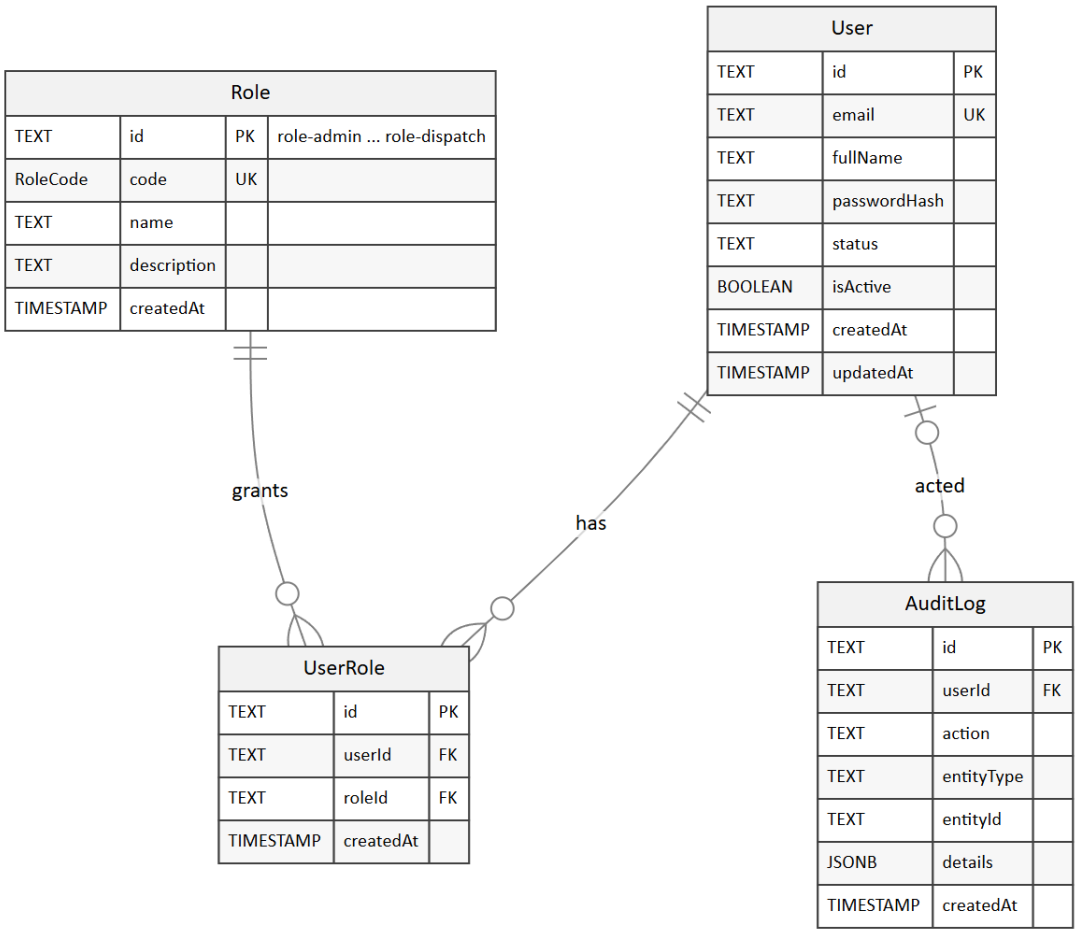


Figure 9: Accounts, roles and the audit log

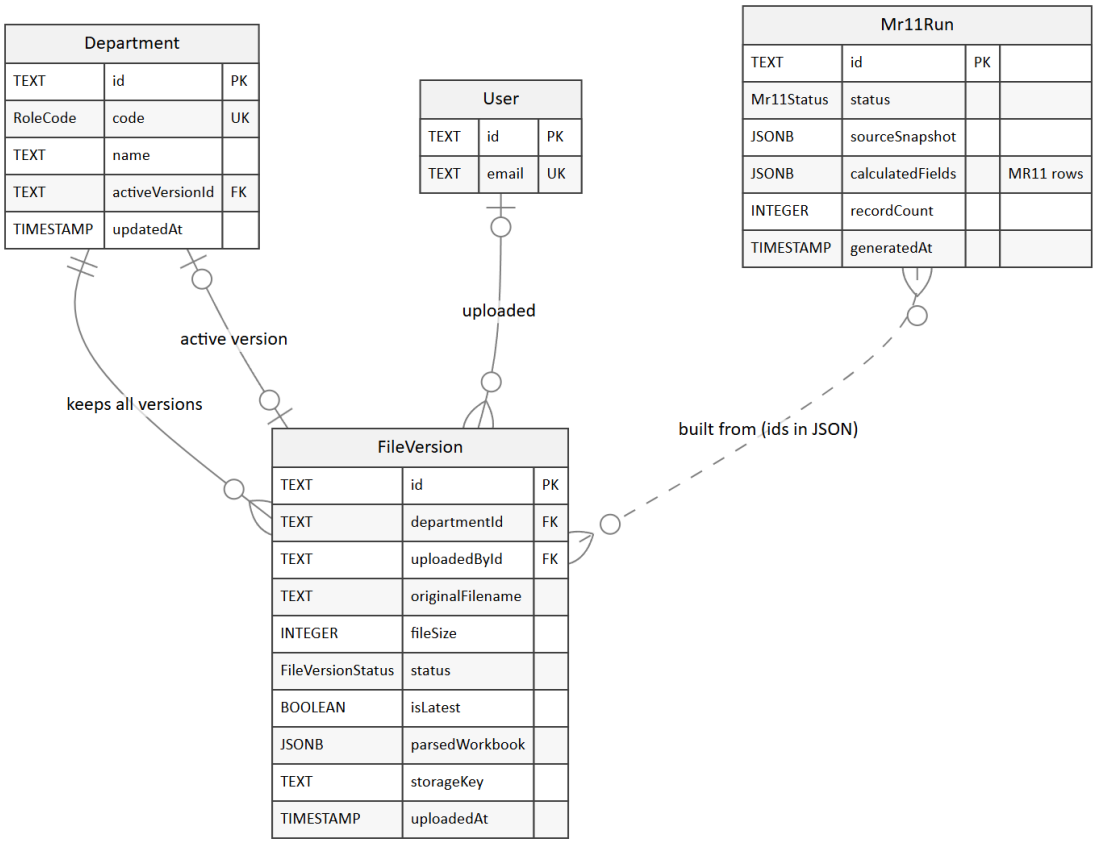


Figure 10: Departments, workbook versions and MR11 runs

Each department points to one active workbook version among all the versions it has kept, and each MR11 run records, inside its JSON snapshot, which workbook versions it was built from. Every column of every table is listed with its data type in Appendix A.

7.1 Tables the application uses

Table	Main columns	Notes
User	id, email (unique), fullName, passwordHash, status, isActive	The password hash is either `pbkdf2$salt$hash` (SHA-512) or, for accounts created in Admin Permissions, bcrypt
Role	id, code (unique), name	The ids must be `role-admin`, `role-ceo`, `role-bd` and so on up to `role-dispatch`
UserRole	userId and roleId, unique as a pair	Links users to roles, many to many
Department	id, code (unique), activeVersionId	Shell Plan & Design's combined workbook is kept under DESIGN
FileVersion	departmentId, originalFilename, storageKey, parsedWorkbook (JSONB), uploadedById, uploadedAt	One row for every upload, never deleted. For uploads made in the browser, `storageKey` is `buffer://<name>`

Table	Main columns	Notes
Mr11Run	status, sourceSnapshot (JSONB), records or calculatedFields (JSONB), recordCount, generatedAt	One row for every build; the newest one is shown

7.2 JSON structures

Structure	Where it is stored	What it holds
Workbook	FileVersion.parsedWorkbook	The sheets, with cell values, font and fill colours, merges and column widths. For Dispatch: `{ sheets, dispatchFiles: { LOCAL, OVERSEAS } }`
MR11 row	Mr11Run.records	The 78 MR11 columns by name, plus internal fields such as `_rowKey`, `_fontColor`, `_fillColor`, `_streamSpan`, `_designDateParts`, `_pricing`, `_dispatchedByMonth` and `_monthStages`
Snapshot	Mr11Run.sourceSnapshot	The workbook version each department contributed, plus `dispatchTracker`, `dispatchByMonth`, `localDispatchByMonth`, `productionByMonth`, `monthStageHistory`, `deletedRows`, `shellplanApprovalHistory` and `engineHistory`

7.3 Tables that exist but are not written to

Table	What happens instead
AuditLog	The audit log is kept in server memory and is lost on restart
Mr11Config	Seeded with an empty column list and never changed
PlanningSeriesHistory, PlanningProjectQuantityTracker and ProductionSeriesHistory	This history is kept in memory and saved inside `Mr11Run.sourceSnapshot.engineHistory`

7.4 Two schema files

The repository describes the same tables in two slightly different layouts: `supabase/schema.sql`, which you run in the Supabase SQL editor and which we recommend, and `prisma/migrations/0_init/migration.sql`. The application only writes the columns a table actually has, so it works with either. Moving to a single layout would still be a good tidy-up.

8. Security

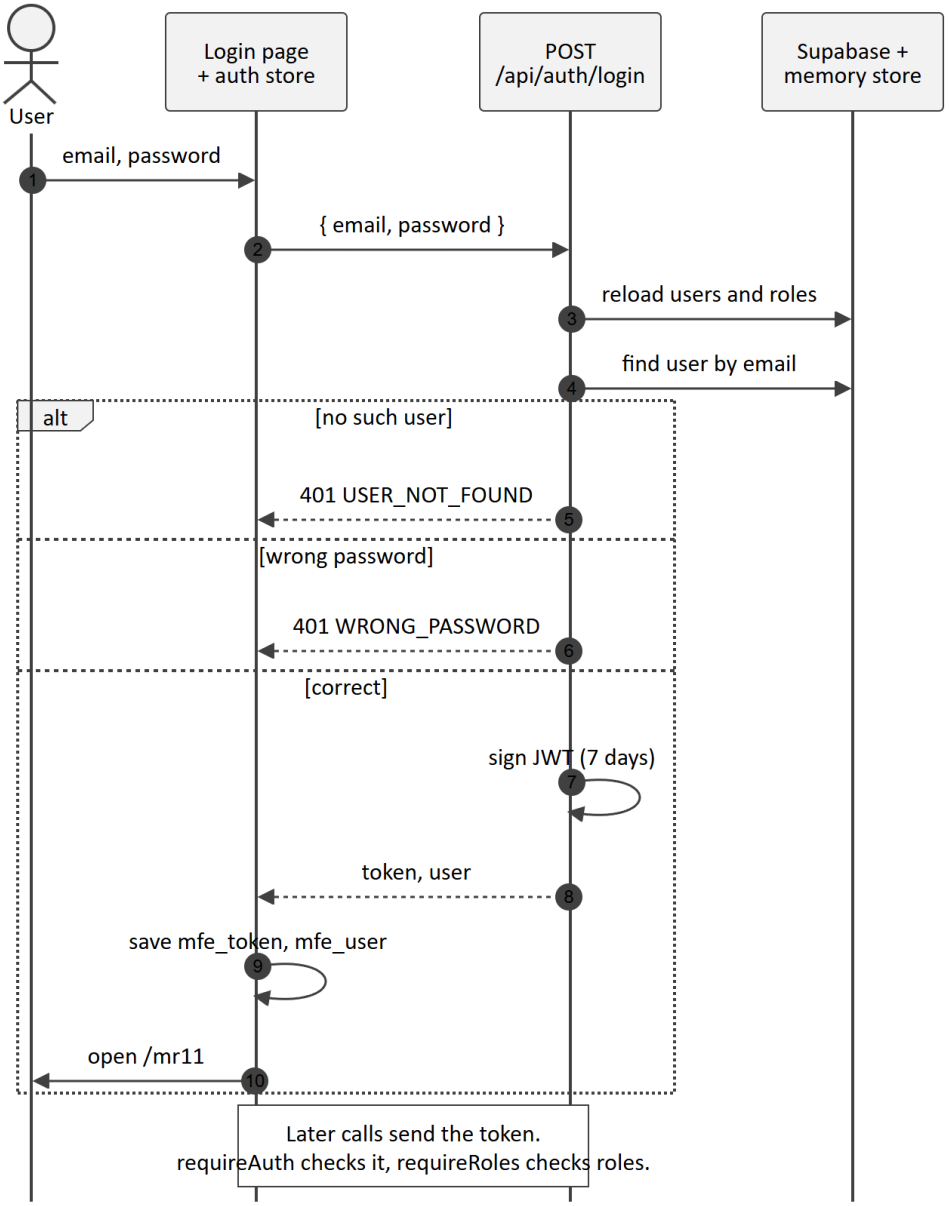


Figure 11: Signing in, and how later requests are checked

Concern	How it is handled
Authentication	Users sign in with an email address and password. The server issues a JWT (HS256), signed with `JWT_SECRET`, that is valid for 7 days
Session storage	The token and user details are kept in the browser's `localStorage` and sent with each request as `Authorization: Bearer`
Authorisation	Middleware on the server checks the roles in the token on every protected route, and ADMIN passes every check. The frontend hides pages using the same rules
Department isolation	A department's workbook can only be read or uploaded by that department (Shell Plan and Design together) or by ADMIN

Concern	How it is handled
Passwords	Hashed with PBKDF2-SHA512 (1,000 iterations) or bcrypt (cost 10)
Transport	HTTPS from Vercel or Azure, and TLS to Supabase. The database certificate is not verified (`sslmode=no-verify`)
Secrets	`DATABASE_URL` and `JWT_SECRET` are kept in the hosting platform's environment settings

The steps needed before go-live are listed in the security hardening checklist in the Operations and Support Guide.

9. Integrations

Service	What it provides	Endpoint	What happens if it fails
Bank Negara Malaysia open API	USD exchange rates (middle rate)	`api.bnm.gov.my/public/exchange-rate`	The system tries open.er-api.com, then Frankfurter (ECB), then the last rate it fetched, and finally `USD_TO_MYR_RATE`
Westmetall	LME aluminium cash and 3-month prices	The LME_Al_cash table on westmetall.com	The last price fetched is reused, and the card shows that it is not live
Supabase PostgreSQL	All lasting data	Transaction pooler, port 6543	Reads are retried once, and failed saves are reported to the user

10. Key design decisions

Decision	Why	What it means
The database is the source of truth and memory is only a cache	Serverless instances can start and stop at any time	Every read or rebuild refreshes from Supabase first, and requests wait for the first load
Departments keep using Excel	No retraining is needed, and their colour conventions keep their meaning	Header names and colours have to stay consistent
Rows are matched by project, stream and nearest colour	Departments use slightly different shades for the same block	Colours within a distance of 100 are treated as the same
Prices are calculated when the MR11 is read	LME prices and exchange rates change every day	The MR11 always shows today's prices, while stored runs keep the inputs
The engine's history is kept inside each MR11 run	Series, approval history, monthly dispatch figures and deleted rows must survive restarts	A single JSON snapshot carries all of this history forward
Writes adapt to the columns a table has	The repository has two table layouts	Either layout works, though moving to one is recommended

11. Environments

Environment	Hosting	Database
Local demonstration	`npm run dev`	None; everything is in memory, using the sample workbooks
Local development	`npm run dev`	Local PostgreSQL on port 5433
Staging (recommended for UAT)	A Vercel preview deployment	A separate Supabase project
Production	Vercel `sin1` or Azure App Service	Supabase `ap-southeast-1`

Appendix A: Data dictionary

This appendix lists every database table, and the columns of the MR11 Master Schedule, with their data types. The database types are the PostgreSQL types created by `supabase/schema.sql`, the install script we recommend. Section A.4 lists the places where the Prisma migration layout is different.

A.1 Enumerated types

Type	Allowed values	Used in
RoleCode	ADMIN, CEO, BD, FINANCE, SHELLPLAN, DESIGN, PLANNING, PRODUCTION, DISPATCH	Role.code and Department.code
Mr11Status	EMPTY, PARTIAL, READY, FAILED	Mr11Run.status
FileVersionStatus	UPLOADING, PROCESSING, READY, FAILED	FileVersion.status

A.2 Tables used by the application

User: the people who can sign in.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key; a UUID, or a fixed seed id such as user-admin-1
email	TEXT	No	None	Unique (User_email_key); stored in lower case
fullName	TEXT	No	None	The name shown on screen
passwordHash	TEXT	No	None	A pbkdf2$salt$hash value or a bcrypt hash
status	TEXT	No	'ACTIVE'	Account status
isActive	BOOLEAN	No	true	
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

Role: the nine roles, which are seeded with fixed ids.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key; role-admin, role-ceo, role-bd and so on up to role-dispatch
code	RoleCode	No	None	Unique (Role_code_key)
name	TEXT	No	None	
description	TEXT	Yes	None	
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

UserRole: links users to roles.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
userId	TEXT	No	None	Refers to User.id; unique together with roleId
roleId	TEXT	No	None	Refers to Role.id
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

Department: the seven departments and their active workbook.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key; dept-bd, dept-finance and so on
code	RoleCode	No	None	Unique (Department_code_key)
name	TEXT	No	None	
description	TEXT	Yes	None	
activeVersionId	TEXT	Yes	None	The FileVersion.id of the workbook in use
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

FileVersion: every workbook that has been uploaded.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key; a UUID
departmentId	TEXT	No	None	Refers to Department.id
uploadedById	TEXT	No	None	Refers to the User.id of the uploader
originalFilename	TEXT	No	None	For Dispatch, in the form "Local: ... / Overseas: ..."
storagePath	TEXT	No	None	The same value as storageKey
fileSize	INTEGER	No	None	In bytes
status	FileVersionStatus	No	'PROCESSING'	The application saves READY
isLatest	BOOLEAN	No	true	Set to false on the department's older versions
sheetMetadata	JSONB	Yes	None	Not used
rawDataJson	JSONB	Yes	None	A copy of parsedWorkbook
errorMessage	TEXT	Yes	None	Not used
uploadedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
storageKey	TEXT	Yes	None	buffer://<file name> for uploads made in the browser
mimeType	TEXT	Yes	None	

Column	Data type	Null	Default	Key and notes
parsedWorkbook	JSONB	Yes	None	The workbook as cell data with its formatting
processedAt	TIMESTAMP(3)	Yes	None	
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

Mr11Run: every build of the MR11; the newest one is the one shown.

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
generatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	Used to find the latest run
generatedById	TEXT	Yes	None	Not used
status	Mr11Status	No	'EMPTY'	The application saves READY
sourceSnapshot	JSONB	Yes	None	The workbook versions used, plus the history carried forward
calculatedFields	JSONB	Yes	None	The MR11 rows (see A.5)
summaryMetrics	JSONB	Yes	None	Not used
recordCount	INTEGER	No	0	The number of MR11 rows

A.3 Tables that version 1.0 does not write to

Mr11Config

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key; a single row called 'singleton'
visibleColumns	TEXT[]	Yes	None	Seeded empty
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedById	TEXT	Yes	None	

AuditLog (kept in server memory instead)

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
userId	TEXT	No	None	Refers to User.id
action	TEXT	No	None	For example UPLOAD_WORKBOOK
entityType	TEXT	No	None	For example DEPARTMENT
entityId	TEXT	Yes	None	
details	JSONB	Yes	None	
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

PlanningSeriesHistory (kept inside Mr11Run.sourceSnapshot instead)

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
projectNo	TEXT	No	None	Unique together with stream, fontColor and seriesNumber
projectShortname	TEXT	Yes	None	
stream	TEXT	Yes	None	
fontColor	TEXT	Yes	None	A hex colour such as #000000
seriesNumber	INTEGER	No	None	
totalProcessed	DOUBLE PRECISION	Yes	None	In m²
detectedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
sourceFileVersionId	TEXT	Yes	None	

PlanningProjectQuantityTracker (kept inside Mr11Run.sourceSnapshot instead)

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
projectNo	TEXT	No	None	Unique together with stream and fontColor
projectShortname	TEXT	Yes	None	
stream	TEXT	Yes	None	
fontColor	TEXT	Yes	None	
totalTargetQuantity	DOUBLE PRECISION	No	None	In m²
totalProcessed	DOUBLE PRECISION	No	0	In m²
balanceQuantity	DOUBLE PRECISION	No	0	In m²
sourceFileVersionId	TEXT	Yes	None	
createdAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	

ProductionSeriesHistory (kept inside Mr11Run.sourceSnapshot instead)

Column	Data type	Null	Default	Key and notes
id	TEXT	No	None	Primary key
projectShortname	TEXT	No	None	Unique together with stream, fontColor and seriesNumber
stream	TEXT	Yes	None	
fontColor	TEXT	Yes	None	
seriesNumber	INTEGER	No	None	
totalFabricated	DOUBLE PRECISION	Yes	None	In m²
detectedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
updatedAt	TIMESTAMP(3)	No	CURRENT_TIMESTAMP	
sourceFileVersionId	TEXT	Yes	None	

A.4 Differences in the Prisma migration layout

`prisma/migrations/0_init/migration.sql` creates the same tables with the differences below. The application works with either layout.

Table	How the Prisma layout differs
All tables	updatedAt has no default. Foreign keys are declared: UserRole to User and to Role (deleted together); FileVersion to Department (deleted together); FileVersion.uploadedById and AuditLog.userId to User (set to null when the user is deleted); Department.activeVersionId to FileVersion (set to null)
Department	activeVersionId must also be unique
FileVersion	There are no storagePath, isLatest, sheetMetadata, rawDataJson or errorMessage columns, and there is an extra checksumSha256 TEXT column. storageKey is NOT NULL, fileSize and uploadedById may be null, and status defaults to 'READY'
Mr11Run	The rows are in records (JSONB, NOT NULL) instead of calculatedFields, sourceSnapshot is NOT NULL, and there are no generatedById or summaryMetrics columns
Mr11Config	visibleColumns is JSONB and NOT NULL, id defaults to 'singleton', and there is no updatedById column
AuditLog	details is called metadata, and userId may be null
PlanningSeriesHistory	Adds projectName TEXT, totalQuantity DOUBLE PRECISION and closingDate TEXT. stream and fontColor are NOT NULL (fontColor defaults to '#000000'), totalProcessed is NOT NULL with a default of 0, and there is no sourceFileVersionId
PlanningProjectQuantityTracker	Has projectName TEXT, lastQuantity DOUBLE PRECISION (NOT NULL, default 0) and lastChangedDate TEXT (NOT NULL) instead of the three quantity columns. stream and fontColor are NOT NULL

Table	How the Prisma layout differs
ProductionSeriesHistory	Adds projectNo TEXT and projectName TEXT, uses totalProduced DOUBLE PRECISION (NOT NULL, default 0) instead of totalFabricated, and makes stream and fontColor NOT NULL

A.5 MR11 Master Schedule columns

Each MR11 row is stored as a JSON object inside Mr11Run. Text values are JSON strings, numbers are JSON numbers, and dates are text in the form YYYY-MM-DD. These are the 78 columns, in the order they appear on screen.

#	Column	Data type	Source	Notes
1	Customer & Project Name	Text	BD	
2	Project No	Text	BD	The 5-digit project number
3	Short Name	Text	BD	The main key used for matching
4	Stream	Text	BD	1, 2, 3 and so on; it may come from the project number
5	Building Name	Text	BD	
6	Countries	Text	BD	Decides between Local and Overseas dispatch
7	Location	Text	BD	The city or site
8	PIC	Text	BD	The person in charge
9	Status	Text	BD	
10	Products type	Text	BD	
11	Formwork type	Text	BD	
12	Remarks	Text	BD	
13	PO	Text	BD	
14	PO date	Date	BD	
15	NCA	Text	BD	
16	NCA date	Date	BD	
17	Original NCA Qty	Number	BD	
18	NCA Remarks	Text	BD	BD's second Remarks column
19	Payment terms - Percentage	Number	BD	A percentage
20	Payment terms - Amount	Number	BD	USD, 2 decimal places
21	Payment terms - Type	Text	BD	
22	Payment terms - Balance Percentage	Number	BD	A percentage
23	Payment terms - Amount 2	Number	BD	USD, 2 decimal places

#	Column	Data type	Source	Notes
24	Payment terms - Type 2	Text	BD	
25	Payment terms - Balance Percentage 2	Number	BD	A percentage
26	Payment terms - Amount 3	Number	BD	USD, 2 decimal places
27	Payment terms - Type 3	Text	BD	
28	Selling Price (USD)	Number	BD, then calculated	USD, MYR and any other currency, at today's rates
29	LME	Text	BD	Fixed, Freeze or Variable
30	LME Rate (USD)	Number	BD	USD per tonne, 3 decimal places
31	Incoterms	Text	BD	
32	Props, WPB, Waler, Acc (USD)	Number	BD	USD
33	Aluminium Weight Adjusted (USD)	Number	BD	USD
34	LME Adjusted (USD)	Number, or the text "Check"	Calculated	3 decimal places; "Check" for Freeze rows
35	Freight Adjusted (USD)	Number	BD	USD
36	Final Selling Price (USD)	Number	Calculated	3 decimal places
37	Advance Received / Payment Status	Text	Finance	
38	Actual Received	Number	Finance	
39	Payment Date	Date	Finance	
40	Shell Plan Status - Pending Consultant Drawings	Text	Shell Plan	In Progress, Pending Drawings, Reapprove or Approved
41	Shell Plan Approved Date	Date	Shell Plan	
42	Formwork Design Status	Text	Design	Fully Complete, Partially Complete, Ongoing or To Start
43	Actual Formwork Order Completion Date	Text (one or more dates)	Design	A single date, or one date per level, such as "Typical: 2026-11-04"
44	Total Quantity Ordered m2	Number	Design	m²
45	Total Processed	Number	Planning	m²; the sum of Cumulative Processed (Project)
46	Processed Date	Date	Calculated	The upload date of the last change
47	Total Produced	Number, or "-"	Production	m²; "-" for Re-Design Only rows
48	Produced Date	Date, or "-"	Production	The latest daily column with output
49	Total Dispatch	Number, or "-"	Dispatch	m²; the sum of Cumulative Dispatched (Project)
50	Dispatch Date	Date, or "-"	Dispatch	
51	Formwork Quantity Sailed (m2)	Number, or "-"	Dispatch	m²

#	Column	Data type	Source	Notes
52	ETD/ATD	Text over several lines	Dispatch	Lines starting "ETD:", "ATD:" or "Delivered:", with dates and m²
53 to 64	Jan-26 to Dec-26	Number	BD and Dispatch	m² for each month: actual for months that have ended, forecast after that
65	Total 2026 m2	Number	Calculated	The sum of the 2026 months
66 to 77	Jan-27 to Dec-27	Number	BD and Dispatch	m² for each month
78	Total 2027 m2	Number	Calculated	The sum of the 2027 months

Each row also carries internal fields, whose names start with an underscore, that are not shown on screen: _rowKey (text), _fontColor and _fillColor (hex text), _streamSpan (a number), _designDateParts (a list of level, date and completed flag), _pricing (the BD price inputs), _dispatchedByMonth (m² for each month) and _monthStages (figures for the CEO pipeline).